UNIVERSIDAD DE CARABOBO

Facultad Experimental de Ciencias y Tecnología

Departamento de Computación Fundamentos de Programación

SISTEMA DE SIMULACIÓN NAVIERA:

FERRYS MARGARITA

Simulador de Control de Carga y Operaciones v1.0

Nombres y Apellidos: Saul Lara, Gabriel Rosario

Cédulas de Identidad: V-31.456.395, V-31.165.402

Sección: 06

Materia: Fundamentos de Programación

Fecha de Entrega: 26 de marzo de 2026

1. Descripción del problema:

Explicación general: El software consiste en un simulador de operaciones navieras para la empresa "El Margariteño C.A.". Su objetivo principal es gestionar el proceso de carga, zarpe y flujo de vehículos y pasajeros en tres ferrys distintos (Lilia Concepción, La Isabela y La Margariteña), optimizando el uso de la rampa de carga y respetando las restricciones de peso y capacidad de cada embarcación.

Contexto y propósito: El programa busca automatizar la logística diaria, clasificando vehículos por tipo (particulares, carga y emergencias) y ruta (Express o Tradicional). El propósito final es generar estadísticas precisas sobre vehículos transportados, ingresos generados y eficiencia operativa del puerto.

2. Análisis de la solución:

Organización de datos: Los datos se gestionan mediante una arquitectura basada en **estructuras (structs)** y **arreglos estáticos,** el sistema utiliza arreglos de tamaño fijo que operan bajo una lógica de cola secuencial. Esto garantiza que el flujo de vehículos sea cronológico, gestionando la prioridad de las emergencias mediante el desplazamiento selectivo de los elementos dentro del índice del arreglo. La información de entrada se lee desde un archivo externo ***(proy1.in)***, se procesa minuto a minuto en un ciclo que simula un día completo (reloj universal), y los resultados se exportan a un archivo de salida ***(proy1.out).***

Justificación de decisiones:

Modularidad: Se utilizaron funciones específicas para tareas como la conversión de tiempo, carga de archivos y manejo de emergencias para facilitar el mantenimiento.

Priorización de emergencias: Se implementó una lógica de inserción al inicio de la cola para ambulancias, bomberos y policías, cumpliendo con protocolos de seguridad reales.

Simulación en tiempo real: El uso de un "tiempo universal" de 0 a 1439 minutos permite modelar con precisión los tiempos de espera y el retorno de los barcos al muelle.

3. Estructuras de datos:

Definición de structs:

vehiculos: Almacena placa, tipo, peso, procedencia, número de pasajeros (adultos/tercera edad) y tipo de pasaje adquirido.

ferrys: Gestiona el estado del barco (Carga, Viaje o Espera), capacidad máxima de peso y vehículos, pasajeros a bordo y un inventario interno de los vehículos cargados.

Arreglos aplicados:

vector_vehiculos[MAX_VEHICULOS]: Lista global de arribos leídos del archivo.

vector_colaExpress y vector_colaTradicional: Colas de espera que actúan como buffers antes de que los vehículos ingresen al ferry.

vector_ferrys[MAX_FERRYS]: Arreglo que contiene las tres unidades de la flota.

4. Algoritmos Implementados:

A continuación, se describen los procesos lógicos de alto nivel que rigen el funcionamiento del simulador naviero "Ferrys Margarita".

4.1 Algoritmo de Gestión de Arribos y Priorización:

Este algoritmo se encarga de recibir los datos del inventario de vehículos y organizar su entrada al sistema de muelles:

Ordenamiento Cronológico: Se aplica un algoritmo de ordenamiento por burbuja sobre el vector principal de vehículos para asegurar que el procesamiento de eventos siga una línea de tiempo real basada en el minuto de llegada.

Clasificación de Colas: Dependiendo de la preferencia del usuario y el tipo de unidad, los vehículos se segmentan en dos estructuras de datos independientes: vector_colaExpress y vector_colaTradicional.

Protocolo de Emergencia: Al detectar vehículos de tipo 5, 6 o 7 (Ambulancia, Bomberos o Policial), se activa una función de inserción con prioridad que desplaza los elementos de la cola para posicionar la emergencia en la cabecera (índice 0), garantizando su atención inmediata.

4.2 Máquina de Estados de la Flota:

Cada ferry en la simulación opera bajo un modelo de estados finitos que controla su disponibilidad en el muelle:

Estado de Espera (3): El ferry permanece inactivo hasta que el reloj universal iguala su tiempo de retorno (Hora de salida + Tiempo de viaje).

Estado de Carga (1): El ferry se vincula a la rampa de acceso y comienza a admitir vehículos de su cola respectiva.

Estado de Viaje (2): Una vez cumplidas las condiciones de zarpe, el ferry se retira del muelle, bloqueando su capacidad de carga hasta completar el trayecto de ida y vuelta.

4.3. Algoritmo de Carga y Desalojo Selectivo:

Es el componente encargado de validar la seguridad y logística del embarque:

Validación de Restricciones Físicas: Antes de confirmar el ingreso de una unidad, el algoritmo verifica que la suma del peso actual, el número de pasajeros y las dimensiones del vehículo no excedan los límites estructurales definidos para el buque (peso_maximo, cap_pasajeros, cap_max_vehiculos).

Algoritmo de Sacrificio por Emergencia: En situaciones donde una unidad de emergencia requiere embarcar y el ferry ha alcanzado su límite, se ejecuta un proceso de desalojo (LIFO - *Last In, First Out* sobre vehículos convencionales). El último vehículo civil cargado es devuelto a la cola de espera y sus valores son restados del total del ferry para ceder el espacio a la autoridad competente.

4.4. Lógica de Decisión de Zarpe:

El sistema evalúa minuto a minuto las condiciones operativas para autorizar la salida de un ferry:

Saturación: Cuando se alcanza la capacidad máxima de vehículos o el límite de pasajeros permitido.

Incompatibilidad de Carga: Si el siguiente vehículo en espera excede el peso máximo permitido para el viaje actual, obligando al ferry a partir para no detener el flujo.

Eficiencia Operativa: Si la cola está vacía pero el ferry ha superado su capacidad mínima de rentabilidad (establecida en el 30% de su capacidad total).

Cierre de Operaciones: Al alcanzar el límite del tiempo universal (11:59 PM), cualquier ferry con carga pendiente recibe la orden de zarpe final para concluir la jornada.

5. Pruebas realizadas:

El código incluye validaciones específicas que sirven como base para los siguientes casos de prueba:

Validación de Integridad del Archivo de Entrada: Se implementó una prueba para verificar que el archivo proy1.in no esté vacío y contenga exactamente los 3 IDs de los ferrys en la primera línea.

Restricción de IDs de Flota: El sistema valida que los IDs de los ferrys sean exclusivamente 1, 2 y 3, y que no existan duplicados en la secuencia de entrada.

Filtrado de Datos Corruptos: Se realizan pruebas de "limpieza" donde el programa ignora automáticamente vehículos con:

Tipos de vehículo fuera del rango 1-7.

Horas de llegada inválidas (mayores a 23:59 o con minutos superiores a 59).

Placas con longitud incorrecta (menores a 3 o mayores a 10 caracteres).Pesos iguales o menores a 0.

Detección de Placas Duplicadas: Se incluyó un algoritmo de búsqueda que evita procesar dos veces el mismo vehículo si la placa ya existe en el sistema.

Simulación de Prioridad (Emergencias): Casos donde vehículos tipo 5, 6 o 7 (Ambulancia, Bomberos, Policía) llegan a una cola llena, verificando su inserción en la posición 0.

5.1. A través de la ejecución del simulador, se observaron los siguientes comportamientos lógicos:

Gestión de Rampa y Tiempos: El sistema respeta un tiempo de carga de **3 minutos** por vehículo bloqueando la rampa mediante la variable rampa_libre para evitar cargas simultáneas.

Eficiencia en el Desalojo por Emergencia: Cuando una emergencia llega y el ferry está lleno, el algoritmo hacer_espacio_emergencia identifica correctamente al último vehículo "civil" (tipo < 5), lo devuelve a la cola y cede el espacio al vehículo prioritario.

Zarpe Automatizado: Los ferrys inician su viaje de manera autónoma al cumplir cualquiera de estos criterios: capacidad máxima de vehículos, límite de pasajeros, exceso de peso con el siguiente vehículo en cola o por fin de la jornada laboral (minuto 1439).

Conversión de Unidades de Peso: El programa normaliza el peso de los vehículos livianos (dividido entre 1000) mientras mantiene el peso real para los vehículos de carga (tipo 4), garantizando que los cálculos de hundimiento y estabilidad sean precisos.

Recuperación de Flota: Los ferrys cambian su estado de "Viaje" a "Espera" automáticamente tras cumplir su tiempo de ciclo (35 o 65 minutos), regresando al muelle para una nueva carga.

6. Conclusiones:

Reflexiones sobre la Experiencia de Desarrollo:

El desarrollo del simulador "Ferrys Margarita" representó un desafío integral que trascendió la simple escritura de código en lenguaje C, permitiendo una inmersión profunda en la resolución de problemas de logística y gestión de recursos. A lo largo del proceso, se identificaron los siguientes aprendizajes clave:

Gestión de la Complejidad y Estructuras de Datos:

La transición de manejar variables aisladas a utilizar Arreglos de Estructuras (struct) fue fundamental para representar entidades complejas como buques y vehículos. Entender que un "Ferry" no es solo un nombre, sino un conjunto de estados, capacidades y un inventario dinámico de carga, permitió construir un sistema escalable y organizado.

El Reloj Universal como Motor de Simulación:

Uno de los mayores retos fue la implementación del tiempo universal en minutos. Esta decisión técnica fue crucial para sincronizar eventos asíncronos (como la llegada aleatoria de vehículos y el retorno de los barcos tras sus viajes). La lógica de "minuto a minuto" permitió que el programa se sintiera como un sistema vivo y no como una simple ejecución lineal.

Ética y Priorización en la Programación:

La implementación de los protocolos de emergencia (ambulancias y cuerpos policiales) aportó una capa de realismo social al proyecto. Desarrollar el "Algoritmo de Desalojo" enseñó la importancia de programar jerarquías y excepciones; en la vida real, el software debe ser capaz de tomar decisiones críticas basadas en la prioridad de la vida sobre la comodidad.

Depuración y Manejo de Archivos:

El trabajo con archivos de entrada (.in) y salida (.out) reforzó la necesidad de una validación de datos rigurosa. Aprender a anticipar errores del usuario (como placas duplicadas, pesos negativos o formatos de hora inválidos) fue una lección sobre la robustez que debe tener cualquier software profesional para evitar fallos críticos en tiempo de ejecución.

6.2. Dificultades encontradas y soluciones:

Gestión vital del Tiempo y Retorno de Flota:

Se solucionó el problema de los ferrys que "se perdían en el mar" debido a comparaciones de tiempo excesivamente rígidas.

Implementación: En la función reiniciar_tiempo_de_viaje, se sustituyó la validación de igualdad estricta por una de **desigualdad compuesta**:

(tiempo_actual >= tiempo_regreso).

Solución Técnica: Esto garantiza que, si por alguna razón el reloj de la simulación salta un minuto o hay un desfase en el procesamiento, el ferry detecte que su viaje terminó y cambie su estado a **"Espera" (Estado 3)** de inmediato.

Máquina de Estados y Control de Rampa:

Se eliminó la dependencia de turnos fijos que bloqueaban el muelle si un ferry no estaba presente.

Implementación: Se centralizó la lógica en la función procesar_carga_ferry.

Solución Técnica: Se introdujo la variable booleana rampa_libre y el manejo de **estados de los ferrys** (1=Carga, 2=Viaje, 3=Espera). Esto permite que el sistema siga evaluando la disponibilidad de la infraestructura física del puerto independientemente de si un barco específico está viajando o no.

Optimización de las Reglas de Zarpe:

Se corrigió el estancamiento de ferrys con baja ocupación mediante una lógica de decisión más inteligente en la función debe_zarpar.

Implementación: El ferry ahora decide zarpar no solo por estar lleno, sino por **incompatibilidad de carga** o **eficiencia mínima**.

Solución Técnica: Se añadieron tres validaciones clave:

Si el siguiente vehículo en cola excede el peso máximo permitido para ese viaje.

Si la cola está vacía pero ya se alcanzó el **30% de la capacidad** (mínimo requerido).

Si se alcanza el límite de pasajeros o vehículos antes de lo previsto.

Gestión Atómica de Emergencias (Protocolo de Desalojo):

Se sustituyeron los bucles de espera infinitos, que causaban bloqueos del programa (*deadlocks*), por un sistema de reordenamiento activo.

Implementación: Se creó la función hacer_espacio_emergencia.

Solución Técnica: Cuando llega una unidad prioritaria (tipo 5, 6 o 7) y no hay espacio, el sistema busca el último vehículo de tipo "civil" (tipo < 5) cargado, lo desaloja del ferry y lo devuelve a la cola en la **posición número dos** (justo detrás de la emergencia). Esto libera espacio de forma inmediata y segura.

Pre-procesamiento y Linealidad de Datos:

Se solucionó el error de "saltos en la cola" donde vehículos que llegaban tarde aparecían antes en la simulación debido a su orden en el archivo de texto.

Implementación: Se incluyó la función ordenar_por_hora al inicio de la ejecución.

Solución Técnica: Mediante un algoritmo de ordenamiento, el vector_vehiculos se organiza por su **tiempo de llegada real** antes de iniciar el reloj universal. Esto asegura que la simulación minuto a minuto sea lineal, coherente y predecible.

6.3. Mejoras en la Lógica de Simulación:

Simulación Multimuelle: El código está diseñado para procesar un ferry a la vez en el muelle principal (rampa_libre). Podría mejorarse la lógica para permitir que varios ferrys carguen simultáneamente si el puerto tuviera múltiples rampas, lo cual requeriría un arreglo de estados de rampa.

Factor de Fallo Aleatorio: Para mayor realismo, se podría introducir una probabilidad de **retraso por clima o falla mecánica**. Esto obligaría al sistema a recalcular los retornos de forma dinámica, probando la robustez de tu función reiniciar_tiempo_de_viaje.

Gestión de Combustible y Costos Operativos: es posible añadir un costo por cada viaje realizado (combustible, mantenimiento, tripulación). Esto permitiría calcular no solo el ingreso bruto, sino la **utilidad neta** al final del día.

Modularización de Tarifas: Actualmente las tarifas están en #define. Sería ideal que el programa leyera un segundo archivo de configuración (ej: tarifas.cfg) para que los precios puedan cambiar sin necesidad de recompilar el código C.

Uso de Memoria Dinámica: Actualmente se utiliza arreglos estáticos con tamaños fijos como MAX_VEHICULOS 1000. Si el archivo de entrada tuviera una exageración de vehículos, el programa fallaría o ignoraría datos. Cambiar a malloc y realloc permitiría que el simulador crezca según la necesidad del archivo.